

PRACA DYPLOMOWA MAGISTERSKA

Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach

*A performance and efficiency analysis of selected
parallel sorting algorithms on multicore processors*

Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Forma studiów: stacjonarne

Poziom studiów: II

Promotor pracy:

dr inż. Bartosz Kowalczyk

Praca przyjęta dnia:

Podpis promotora:

Częstochowa, 2026

Niniejszym chciałbym serdecznie podziękować

...

*za bezcenne wsparcie udzielone mi
w trakcie trwania studiów.*

Spis treści

Wstęp	3
Cel pracy	4
Zakres pracy	4
1 Wprowadzenie teoretyczne	7
1.1 Wprowadzenie do sortowania równoległego	7
1.2 Charakterystyka procesorów wielordzeniowych	7
1.3 Wydajność i efektywność	7
1.4 Skalowalność algorytmów równoległych	7
2 Opis wybranych algorytmów sortowania równoległego	9
2.1 Parallel Quicksort	9
2.2 Parallel Merge Sort	9
2.3 Parallel Bucket Sort	9
2.4 Parallel Samplesort	9
3 Środowisko badawcze i implementacja	11
3.1 Opis środowiska testowego	11
3.2 Wybór języka i bibliotek	12
3.3 Sposób implementacji algorytmów	13
3.3.1 Ogólna architektura rozwiązania	13
3.3.2 Parallel Quicksort	13
3.3.3 Parallel Merge Sort	15
3.3.4 Parallel Bucket Sort	16
3.3.5 Parallel Samplesort	18
3.4 Charakterystyka danych testowych	19
3.4.1 Typy i rozmiary danych	20
3.4.2 Charakterystyka zbiorów	20
3.4.3 Sposób przechowywania	21
4 Metodyka badań	23
4.1 Scenariusze testowe	23
4.2 Wariant sekwencyjny jako punkt odniesienia	23

4.3	Parametry pomiarów	23
4.4	Narzędzia do pomiaru czasu, CPU i pamięci	23
4.5	Przebieg eksperymentów	23
5	Analiza wyników badań	25
5.1	Porównanie czasu sortowania	25
5.2	Wpływ liczby rdzeni na wydajność	25
5.3	Analiza zużycia pamięci RAM i obciążenia CPU	25
5.4	Skalowalność algorytmów	25
5.5	Wpływ typu danych	25
5.6	Porównanie efektywności w zależności od charakterystyki danych wej- ściowych	26
	Podsumowanie	27
	Bibliografia	29
	Spis rysunków	31
	Spis tabel	32
	Listing	33
	Streszczenie	35
	Summary	37
	Słowa kluczowe	39

Wstęp

W ostatnim czasie można zaobserwować intensywny rozwój architektury procesorów wielordzeniowych. Producenci sprzętu zamiast dalszego zwiększania częstotliwości taktowania pojedynczego rdzenia decydują się na zwiększanie liczby rdzeni fizycznych oraz logicznych. Taka zmiana podejścia projektowania procesorów stawia przed twórcami oprogramowania nowe wyzwania. Klasyczne algorytmy sekwencyjne nie wykorzystują w pełni dostępnej mocy obliczeniowej współczesnych jednostek centralnych.

Szczególnie ważnym obszarem, w którym równoległość może przynieść wymierne korzyści, jest sortowanie danych. Sortowanie stanowi jeden z fundamentalnych problemów informatyki i jest wykorzystywane w ogromnej liczbie miejsc takich jak bazy danych, systemy plików, aż po przetwarzanie dużych zbiorów danych i uczenie maszynowe. Wraz ze wzrostem objętości przetwarzanych informacji klasyczne algorytmy sortowania sekwencyjnego mogą coraz częściej stać się wąskim gardłem wydajnościowym.

Niestety, implementacja efektywnych algorytmów sortowania równoległego nie jest zadaniem łatwym. Wymaga ona odpowiedniego podziału danych, synchronizacji procesów, zarządzania pamięcią współdzieloną oraz minimalizacji narzutu związanego z tworzeniem i komunikacją procesów. W praktyce wiele istniejących rozwiązań albo pozostaje na poziomie teoretycznym, albo nie uwzględnia realnych ograniczeń sprzętowych i systemowych, takich jak koszt przełączania kontekstu, konflikty w dostępie do pamięci czy nierównomierne obciążenie rdzeni.

W związku z tym, interesującym i aktualnym zagadnieniem wydaje się praktyczne zbadanie wydajności oraz efektywności wybranych algorytmów sortowania równoległego. Niniejsza praca koncentruje się na implementacji i porównaniu czterech popularnych algorytmów: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort. Algorytmy zostały zrealizowane w języku Python z wykorzystaniem biblioteki `multiprocessing`, a następnie poddane testom pod kątem czasu wykonania, zużycia pamięci RAM, obciążenia procesora, skalowalności oraz wpływu charakterystyki danych wejściowych.

Cel pracy

Głównym celem niniejszej pracy magisterskiej jest przeprowadzenie dogłębnej analizy wydajności i efektywności wybranych algorytmów sortowania równoległego na współczesnych wielordzeniowych procesorach. Praca skupia się na praktycznej implementacji oraz porównaniu czterech algorytmów: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort.

Kluczowym celem jest zbadanie, w jaki sposób wybrane algorytmy zachowują się w realnych warunkach obliczeniowych, a nie jedynie w warunkach idealnych zakładanych przez modele teoretyczne. W tym celu algorytmy zostały zaimplementowane w języku Python z wykorzystaniem biblioteki `multiprocessing`, która umożliwia tworzenie procesów oraz współdzielenie pamięci. Każdy z algorytmów został uruchomiony zarówno w wariancie sekwencyjnym stanowiącym punkt odniesienia, jak i w wariancie równoległym, przy różnej liczbie wykorzystywanych rdzeni procesora.

Badanie obejmuje kilka kluczowych aspektów:

- ▶ pomiar czasu sortowania w zależności od rozmiaru zbioru danych oraz liczby użytych rdzeni,
- ▶ analizę zużycia pamięci operacyjnej (RAM) oraz obciążenia procesora (CPU) podczas wykonywania algorytmów,
- ▶ ocenę skalowalności, czyli zdolności algorytmów do efektywnego wykorzystania rosnącej liczby rdzeni,
- ▶ porównanie zachowania algorytmów przy różnych charakterystykach danych wejściowych tj. dane losowe, częściowo posortowane oraz z duplikatami.

Dodatkowym celem pracy jest wyznaczenie i analiza klasycznych metryk równoległości, *speedup* (przyspieszenie) oraz *efficiency* (efektywność), a także identyfikacja głównych czynników ograniczających wydajność poszczególnych algorytmów (m.in. narzut związany z tworzeniem procesów, synchronizacją, komunikacją oraz zarządzaniem pamięcią współdzieloną).

W rezultacie pracy oczekuje się uzyskania praktycznych wniosków dotyczących tego, które z badanych algorytmów sortowania równoległego najlepiej sprawdzają się w środowisku wielordzeniowym przy określonych rozmiarach i typach danych, a także wskazania ich mocnych i słabych stron w kontekście rzeczywistego wykorzystania zasobów obliczeniowych.

Zakres pracy

Zakres pracy obejmuje:

- ▶ zebranie wiadomości z zakresu sortowania równoległego, architektury procesorów wielordzeniowych, metryk wydajności i efektywności oraz skalowalności algorytmów równoległych,
- ▶ porównanie wybranych algorytmów sortowania równoległego (Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort) pod kątem wydajności, skalowalności i zużycia zasobów obliczeniowych,
- ▶ opracowanie założeń projektowych dotyczących implementacji algorytmów w środowisku wielordzeniowym z wykorzystaniem biblioteki `multiprocessing` oraz pamięci współdzielonej,
- ▶ implementację wariantów sekwencyjnych i równoległych wybranych algorytmów sortowania w języku Python,
- ▶ implementację generatora danych testowych o różnych charakterystykach (dane losowe, częściowo posortowane, oraz dane z duplikatami),
- ▶ przetestowanie zaimplementowanych algorytmów na wielordzeniowych procesorach, obejmujące pomiar czasu wykonania, zużycia pamięci RAM, obciążenia CPU, wyznaczenie metryk *speedup* i *efficiency* oraz analizę wpływu charakterystyki danych wejściowych.
- ▶ prezentację wyników badań za pomocą tabel oraz wykresów.

W rozdziale 1 przedstawiono podstawy teoretyczne sortowania równoległego, charakterystykę procesorów wielordzeniowych oraz metryki wykorzystywane do oceny wydajności i skalowalności. W rozdziale 2 opisano wybrane algorytmy sortowania równoległego. W rozdziale 3 scharakteryzowano środowisko badawcze, zastosowane technologie oraz sposób implementacji algorytmów. W rozdziale 4 przedstawiono metodykę badań, scenariusze testowe oraz narzędzia pomiarowe. W rozdziale 5 zaprezentowano i przeanalizowano wyniki przeprowadzonych eksperymentów. Pracę zamyka podsumowanie zawierające najważniejsze wnioski oraz możliwe kierunki dalszych badań.

Rozdział 1

Wprowadzenie teoretyczne

- 1.1 Wprowadzenie do sortowania równoległego**
- 1.2 Charakterystyka procesorów wielordzeniowych**
- 1.3 Wydajność i efektywność**
- 1.4 Skalowalność algorytmów równoległych**

Rozdział 2

Opis wybranych algorytmów sortowania równoległego

Dział poświęcony wybranym algorytmom zawierający opis, schemat działania, sposób dzielenia i łączenia, złożoność, potencjalne problemy oraz kiedy algorytm jest efektywny, a kiedy nie

2.1 Parallel Quicksort

2.2 Parallel Merge Sort

2.3 Parallel Bucket Sort

2.4 Parallel Samplesort

Rozdział 3

Środowisko badawcze i implementacja

3.1 Opis środowiska testowego

Wszystkie eksperymenty zostały przeprowadzone na komputerze o następującej konfiguracji:

- ▶ **Procesor:** Intel Core i5-9300H @ 2.40 GHz
- ▶ **Rdzenie fizyczne:** 4
- ▶ **Rdzenie logiczne:** 8
- ▶ **Taktowanie bazowe:** 2401 MHz
- ▶ **Pamięć operacyjna RAM:** 15,85 GB
- ▶ **System operacyjny:** Windows 10
- ▶ **Architektura:** AMD64 (x86-64)
- ▶ **Interpreter Python:** 3.11.7

Środowisko to umożliwia badanie zachowania algorytmów sortowania równoległego przy ograniczonej, ale realnej liczbie rdzeni. Obecność technologii Hyper-Threading (8 procesorów logicznych przy 4 rdzeniach fizycznych) pozwala dodatkowo ocenić wpływ logicznych jednostek wykonawczych na wydajność algorytmów.

Dane dotyczące sprzętu i oprogramowania były automatycznie zebrane i zapisywane w bazie danych SQLite przed przeprowadzaniem eksperymentów.

3.2 Wybór języka i bibliotek

Do implementacji algorytmów sortowania równoległego, genrowania danych, przeprowadzenia eksperymentów oraz wizualizacji wyników wybrano język programowania Python w wersji 3.11.7 [11]. Decyzja ta wynikała z kilku powodów. Python jest jednym z najpopularniejszych języków, charakteryzuje się przejrzystą i elegancką składnią, co ułatwia tworzenie, analizę i utrzymanie kodu. Posiada bogatą bibliotekę standardową oraz rozbudowany zbiór dostępnych bibliotek, pakietów i narzędzi programistycznych rozszerzających możliwości języka, które znacząco przyspieszają rozwój oprogramowania. Dodatkowo język ten jest w pełni przenośny między systemami operacyjnymi tj. Windows, Linux, macOS, co ma znaczenie przy planowaniu ewentualnych dalszych testów na innych platformach.

Niezwyczajnie istotną rolę w projekcie odgrywa biblioteka `multiprocessing` [12], stanowiąca część standardowej biblioteki Pythona. Pakiet ten przy użyciu interfejsu programistycznego podobnego do pythonowego modułu `threading` umożliwia tworzenie procesów oraz realizację obliczeń równoległych z pominięciem ograniczeń wynikających z Global Interpreter Lock (GIL). Dzięki temu możliwe jest efektywne wykorzystanie wielu rdzeni procesora. W ramach tej biblioteki wykorzystano również moduł `multiprocessing.shared_memory` [13], który pozwala na bezpośredni dostęp wielu procesów do wspólnego obszaru pamięci. Mechanizm ten eliminuje konieczność serializacji oraz przesyłania dużych struktur danych między procesami, ograniczając związek z tym narzut, co ma szczególne znaczenie podczas sortowania dużych zbiorów danych.

Biblioteka `NumPy` [14] obsługująca dane numeryczne i szybkie operacje na tablicach, została wykorzystana w optymalizacji algorytmów `Parallel Bucket Sort` oraz `Parallel Samplesort`. Posłużyła w nich do efektywnego rozdzielania elementów do kubeków, co znacznie przyspieszyło etap dystrybucji danych. Ponadto `NumPy` wykorzystano w module analizy złożoności obliczeniowej oraz przy generowaniu wykresów i rankingów.

Wyniki eksperymentów były przechowywane i przetwarzane z użyciem biblioteki `pandas` [15], która oferuje wygodne struktury danych (`DataFrame`) oraz narzędzia do filtrowania, agregacji i analizy tabelarycznej. Do generowania wykresów i wizualizacji wyników wykorzystano bibliotekę `matplotlib` [16].

Do monitorowania zużycia zasobów systemowych wykorzystano bibliotekę `psutil` [17] oraz narzędzia `py-cpuinfo` i `py-spy` [18].

Całość kodu źródłowego została opracowana w środowisku programistycznym `JetBrains PyCharm` [19], które zapewnia wsparcie dla języka Python, debugowanie oraz wygodne zarządzanie projektem.

Do przechowywania wygenerowanych danych testowych oraz wyników eksperymentów wykorzystano bazę danych `SQLite` [20]. Jest to wbudowany silnik bazo-

danowy, który nie wymaga osobnego serwera ponieważ odczytuje i zapisuje dane bezpośrednio do plików dyskowych. Rozwiązanie to upraszcza zarządzanie danymi i zapewnia łatwą przenośność między środowiskami.

Wybór powyższego zestawu technologii umożliwia na spójną implementację algorytmów, precyzyjny pomiar ich charakterystyk wydajnościowych oraz wygodną analizę i prezentację wyników badań.

3.3 Sposób implementacji algorytmów

Wszystkie algorytmy zaimplementowano w języku Python z wykorzystaniem mechanizmu wieloprosesowego oraz pamięci współdzielonej. Każdy algorytm występuje w dwóch wariantach: sekwencyjnym, który stanowi punkt odniesienia oraz równoległym. Wspólną cechą implementacji równoległych jest przechowywanie sortowanych danych w bloku pamięci współdzielonej, mapowanym na tablicę typów `ctypes` (`c_longlong` dla liczb całkowitych oraz `c_double` dla liczb zmiennoprzecinkowych). Dzięki temu procesy potomne mogą odczytywać i modyfikować te same dane bez konieczności ich kopiowania oraz przesyłania pomiędzy procesami.

3.3.1 Ogólna architektura rozwiązania

Przy algorytmach opartych na strategii “dziel i zwyciężaj” czyli Parallel Quicksort oraz Parallel Merge Sort, zastosowano rekurencyjne tworzenie procesów z ograniczeniem głębokości równoległości za pomocą parametru `max_depth`. Dodatkowo wprowadzono progi minimalnego rozmiaru podproblemu zawarte w słowniku `PARALLEL_CUTOFF`, poniżej których dalsze tworzenie procesów jest nieopłacalne i następuje przejście do sortowania sekwencyjnego. Takie rozwiązanie ogranicza narzut związany z tworzeniem zbyt dużej liczby procesów przy małych fragmentach danych.

Dla algorytmów opartych na kubekach czyli Parallel Bucket Sort oraz Parallel Samplesort, zastosowano model z góry określonej liczby procesów `process_count`. Dane są najpierw rozdzielane do kubków, a następnie grupy kubków są przydzielane poszczególnym procesom. Również tutaj wprowadzono progi minimalnego rozmiaru zawarte w słownikach `PARALLEL_SIZE_CUTOFF` oraz `GROUP_SIZE_CUTOFF`, poniżej których algorytm przełącza się na wariant sekwencyjny.

3.3.2 Parallel Quicksort

Wariant sekwencyjny realizuje klasyczny Quicksort z wyborem pivota jako elementu środkowego.

```
1 def partition(arr, low, high):  
2     pivot = arr[(low + high) // 2]
```

```

3
4     i = low
5     j = high
6
7     while i <= j:
8         while arr[i] < pivot:
9             i += 1
10        while arr[j] > pivot:
11            j -= 1
12        if i <= j:
13            arr[i], arr[j] = arr[j], arr[i]
14            i += 1
15            j -= 1
16
17    return i

```

Listing 3.1: Funkcja partycjonowania w Quicksort

W wersji równoległej po wykonaniu partycji lewa i prawa część tablicy mogą być sortowane niezależnie przez osobne procesy. Każdy proces dołącza się do istniejącego bloku pamięci współdzielonej, sortuje przypisany dla siebie zakres, a następnie kończy działanie. Gdy rozmiar zakresu spada poniżej ustalonego progu lub osiągnięta zostaje maksymalna głębokość równoległości, dalsze sortowanie odbywa się sekwencyjnie.

```

1  if size <= min_size or depth >= max_depth:
2      sort_in_place_on_shared(arr, low, high)
3      return
4
5  local = arr[low:high + 1]
6  local_index = partition(local, 0, size - 1)
7  arr[low:high + 1] = local
8  index = low + local_index
9
10 left = (low, index - 1)
11 right = (index, high)
12
13 processes = []
14
15 for part in [left, right]:
16     p_low, p_high = part
17     if p_low >= p_high:
18         continue
19
20     part_size = p_high - p_low + 1
21     if part_size > min_size:
22         p = mp.Process(
23             target=parallel_quicksort_worker,
24             args=(shm_name, length, dtype, p_low, p_high,

```

```

25         depth + 1, max_depth, min_size)
26     )
27     p.start()
28     processes.append(p)
29     else:
30         sort_in_place_on_shared(arr, p_low, p_high)
31
32 for p in processes:
33     p.join()

```

Listing 3.2: Tworzenie procesów w Parallel Quicksort

3.3.3 Parallel Merge Sort

Wariant sekwencyjny realizuje klasyczny Merge Sort z rekurencyjnym podziałem tablicy na połowy oraz scalaniem posortowanych części.

```

1 def merge(arr, left, mid, right):
2     temp = [None] * (right - left + 1)
3
4     i = left
5     j = mid + 1
6     k = 0
7
8     while i <= mid and j <= right:
9         if arr[i] <= arr[j]:
10             temp[k] = arr[i]
11             i += 1
12         else:
13             temp[k] = arr[j]
14             j += 1
15
16         k += 1
17
18     while i <= mid:
19         temp[k] = arr[i]
20         i += 1
21         k += 1
22
23     while j <= right:
24         temp[k] = arr[j]
25         j += 1
26         k += 1
27
28     arr[left:right + 1] = temp

```

Listing 3.3: Funkcja scalania w Merge Sort

W wersji równoległej obie połowy mogą być sortowane jednocześnie przez osobne procesy. Po zakończeniu obu procesów następuje etap scalania, który w danej implementacji wykonywany jest sekwencyjnie. Głębokość równoległości oraz minimalny rozmiar zakresu ograniczają nadmierne tworzenie procesów.

```
1 if size <= min_size or depth >= max_depth:
2     sort_in_place_on_shared(arr, left, right)
3     return
4
5 mid = (left + right) // 2
6 processes = []
7
8 for part_left, part_right in [(left, mid), (mid + 1, right)]:
9     part_size = part_right - part_left + 1
10    if part_size > min_size:
11        p = mp.Process(
12            target=parallel_mergesort_worker,
13            args=(shm_name, length, dtype, part_left, part_right,
14                depth + 1, max_depth, min_size)
15        )
16        p.start()
17        processes.append(p)
18    else:
19        sort_in_place_on_shared(arr, part_left, part_right)
20
21 for process in processes:
22     process.join()
23
24 local = arr[left:right + 1]
25 merge(local, 0, mid - left, size - 1)
26 arr[left:right + 1] = local
```

Listing 3.4: Równoległe sortowanie połówek i sekwencyjne scalanie w Parallel Merge Sort

3.3.4 Parallel Bucket Sort

Wariant sekwencyjny rozdziela elementy do liczby kubełków wyznaczonej na podstawie rozmiaru danych, a następnie sortuje każdy kubełek z wykorzystaniem zaimplementowanego algorytmu Merge Sort. Na sam koniec łączy wyniki w jedną posortowaną tablicę.

```
1 def distribute_to_buckets(arr, bucket_count):
2     if len(arr) == 0:
3         return []
4
5     arr_np = np.asarray(arr)
6     min_value = arr_np.min()
```

```

7     max_value = arr_np.max()
8
9     if min_value == max_value:
10         buckets = [[] for _ in range(bucket_count)]
11         buckets[0] = list(arr)
12         return buckets
13
14     bucket_range = (max_value - min_value) / bucket_count
15     indices = np.minimum(
16         bucket_count - 1,
17         ((arr_np - min_value) / bucket_range).astype(np.int64)
18     )
19
20     order = np.argsort(indices, kind='stable')
21     sorted_indices = indices[order]
22     sorted_values = arr_np[order]
23
24     buckets = [[] for _ in range(bucket_count)]
25     boundaries = np.searchsorted(sorted_indices, np.arange(bucket_count +
26         1))
27     for i in range(bucket_count):
28         buckets[i] = sorted_values[boundaries[i]:boundaries[i + 1]].
29         tolist()
30
31     return buckets

```

Listing 3.5: Rozdzielanie elementów do kubeków w Bucket Sort

W wersji równoległej liczba utworzonych kubeków jest wyliczana na podstawie rozmiaru danych oraz liczby wykorzystywanych rdzeni. Po utworzeniu kubeków ich zawartość jest łączona w jedną tablicę, która umieszczana zostaje w pamięci współdzielonej, a zakresy poszczególnych kubeków są grupowane i przydzielane procesom. Procesy sortują przypisane im grupy kubeków niezależnie. Dla małych grup sortowanie wykonywane jest sekwencyjnie, co ogranicza narzut związany z tworzeniem procesów. Do efektywnego rozdzielania elementów wykorzystano operacje wektorowe biblioteki NumPy.

```

1 bucket_groups = split_bucket_ranges(bucket_ranges, process_count)
2 processes = []
3 sequential_groups = []
4
5 for group in bucket_groups:
6     if not group:
7         continue
8
9     group_size = group[-1][1] - group[0][0] + 1
10
11     if should_spawn_for_group(group_size, process_count):

```

```

12         process = mp.Process(
13             target=bucket_worker,
14             args=(shm.name, len(arr), dtype, group)
15         )
16         process.start()
17         processes.append(process)
18     else:
19         sequential_groups.append(group)
20
21 for group in sequential_groups:
22     bucket_worker_inline(arr, group)
23
24 for process in processes:
25     process.join()

```

Listing 3.6: Przydzielanie grup kubełków procesom w Parallel Bucket Sort

3.3.5 Parallel Samplesort

Wariant sekwencyjny działa w kilku etapach: podział danych na części, lokalne sortowanie, wybór próbek, wyznaczenie wartości rozdzielających, rozdzielanie elementów do kubełków według wyznaczonych wartości oraz ostateczne sortowanie kubełków.

```

1 def distribute_to_buckets(data, pivots):
2     if not pivots:
3         return [list(data)]
4
5     arr = np.asarray(data)
6     pivots_arr = np.asarray(pivots)
7
8     indices = np.searchsorted(pivots_arr, arr, side='right')
9
10    order = np.argsort(indices, kind='stable')
11    sorted_indices = indices[order]
12    sorted_values = arr[order]
13    boundaries = np.searchsorted(sorted_indices, np.arange(len(pivots) +
14        2))
15
16    buckets = []
17    for i in range(len(pivots) + 1):
18        buckets.append(sorted_values[boundaries[i]:boundaries[i + 1]].
19            tolist())
20
21    return buckets

```

Listing 3.7: Rozdzielanie elementów do kubełków na podstawie wartości rozdzielających w Samplesort

W wersji równoległej etapy te realizowane są z wykorzystaniem wielu procesów. Na początku procesy równoległe sortują lokalne fragmenty danych i zbierają próbki. Na podstawie próbek wyznaczone są wartości rodzielające, po czym dane elementy są rozdzielane do kubeków. Na końcu grupy kubeków są ponownie sortowane równoległe. W przypadku niewielkich grup danych stosowany jest wariant sekwencyjny, co ogranicza koszt związany z tworzeniem procesów. Do dystrybucji elementów wykorzystano operacje wektorowe biblioteki NumPy.

```
1 bucket_groups = split_bucket_ranges(bucket_ranges, process_count)
2 processes = []
3 sequential_groups = []
4 min_group_size = get_group_size_cutoff(process_count)
5
6 for group in bucket_groups:
7     if not group:
8         continue
9
10    group_size = group[-1][1] - group[0][0] + 1
11
12    if group_size > min_group_size:
13        process = mp.Process(
14            target=bucket_worker,
15            args=(shm.name, len(arr), dtype, group)
16        )
17        process.start()
18        processes.append(process)
19    else:
20        sequential_groups.append(group)
21
22 for group in sequential_groups:
23     sort_group(arr, group)
24
25 for process in processes:
26     process.join()
```

Listing 3.8: Równoległe sortowanie grup kubeków w Parallel Samplesort

3.4 Charakterystyka danych testowych

Do przeprowadzenia testów przygotowano zestawy danych o zróżnicowanej charakterystyce, tak aby możliwe było ocenienie zachowania algorytmów w różnych scenariuszach.

3.4.1 Typy i rozmiary danych

Testy przeprowadzono dla dwóch typów wartości:

- ▶ liczb całkowitych - `int`
- ▶ liczb zmiennoprzecinkowych - `float`

Dla każdego typu wygenerowano zbiory o następujących rozmiarach:

- ▶ 1 000 elementów
- ▶ 10 000 elementów
- ▶ 100 000 elementów
- ▶ 1 000 000 elementów

Wygenerowane wartości znajdują się w zakresie od 0 do 1 000 000.

3.4.2 Charakterystyka zbiorów

Przygotowano trzy główne kategorie danych:

- ▶ **Dane losowe** – elementy generowane w sposób całkowicie losowy z użyciem generatora liczb pseudolosowych.
- ▶ **Dane z duplikatami** – zbiory, w których około 40% elementów stanowią powtórzenia. Najpierw wygenerowano 60% unikalnych wartości zbioru o danym rozmiarze, a następnie na ich podstawie dodano brakującą część poprzez losowe powtórzenia.
- ▶ **Dane częściowo posortowane** – zbiory zawierające odpowiednio 20%, 40%, 60% oraz 80% elementów uporządkowanych. Posortowane fragmenty przeplatano fragmentami losowymi, co pozwoliło uzyskać zbiory o określonym stopniu częściowego uporządkowania.

```
1 def generate_part_sorted_values(table_name, count, scope, sorted_ratio):
2     sorted_count = int(count * sorted_ratio)
3     random_count = count - sorted_count
4     parts = 5
5
6     if "int" in table_name:
7         sorted_values = sorted(random.randint(0, scope) for _ in range(
8             sorted_count))
9         random_values = [random.randint(0, scope) for _ in range(
10             random_count)]
11     else:
```



```

10         sorted_values = sorted(random.uniform(0, scope) for _ in range(
            sorted_count))
11         random_values = [random.uniform(0, scope) for _ in range(
            random_count)]
12
13         chunk_size = sorted_count // parts
14         sorted_chunks = [sorted_values[i*chunk_size:(i+1)*chunk_size] for i
            in range(parts-1)]
15         sorted_chunks.append(sorted_values[(parts-1)*chunk_size:])
16
17         random_chunk_size = random_count // (parts + 1)
18         random_chunks = [random_values[i*random_chunk_size:(i+1)*
            random_chunk_size] for i in range(parts)]
19         random_chunks.append(random_values[parts*random_chunk_size:])
20
21         combined = []
22         for i in range(parts):
23             combined.extend(random_chunks[i])
24             combined.extend(sorted_chunks[i])
25         combined.extend(random_chunks[-1])
26
27         values = [(val, count) for val in combined]
28
29         return values

```

Listing 3.9: Generowanie danych częściowo posortowanych

Do testów przygotowano 12 wariantów zbiorów (2 typy danych X 6 charakterystyk).

3.4.3 Sposób przechowywania

Wszystkie dane testowe zapisano w bazie danych SQLite. Każdy wariant przechowywany był w osobnej tabeli o strukturze:

- ▶ `id` – unikalny identyfikator rekordu,
- ▶ `value` – wartość elementu `INTEGER` lub `REAL`
- ▶ `set_size` – rozmiar zbioru, do którego należy dany element, ograniczony do wartości 1000, 10 000, 100 000, 1000 000

Rozdział 4

Metodyka badań

4.1 Scenariusze testowe

Opis różnych warunków i przypadków testowych takich jak na jakich rozmiar danych, wariant algorytmu, liczba rdzeni, różne typy zbiorów (losowe itp.)

4.2 Wariant sekwencyjny jako punkt odniesienia

Wyjaśnienie, że dla porównania wyników algorytmów równoległych testowana jest także wersja sekwencyjna każdego algorytmu.

4.3 Parametry pomiarów

Parametry pomiarów co i jak będzie mierzone

4.4 Narzędzia do pomiaru czasu, CPU i pamięci

Opis wykorzystanych narzędzi i bibliotek użytych do pomiarów

4.5 Przebieg eksperymentów

Opis jak zostały przeprowadzone testy. Przygotowanie środowisk testowych. Sposób zapisywania i archiwizacji wyników. Procedura uruchomienia algorytmu itp.

Rozdział 5

Analiza wyników badań

5.1 Porównanie czasu sortowania

Prezentacja wyników pomiarów czasu wykonania dla każdego z algorytmów (dla różnych zbiorów danych, rozmiarów i typów danych). Omówienie i analiza

5.2 Wpływ liczby rdzeni na wydajność

Prezentacja i omówienie wyników pod względem użytych liczby rdzeni

5.3 Analiza zużycia pamięci RAM i obciążenia CPU

Prezentacja i omówienie zużycia pamięci i obciążenia w tym zestawienie średniego i maksymalnego zużycia

5.4 Skalowalność algorytmów

Ocena, jak dobrze algorytmy skalują się wraz ze wzrostem liczby rdzeni i rozmiarem danych

5.5 Wpływ typu danych

Porównanie wyników dla tych samych algorytmów, ale różnych typów danych wejściowych

5.6 Porównanie efektywności w zależności od charakterystyki danych wejściowych

Omówienie wyników dla danych losowych itp. Wskazanie, który algorytm radził sobie lepiej przy danym rodzaju danych

Podsumowanie

Podsumowanie wyników, najważniejsze wnioski, ocena czy cele pracy zostały osiągnięte, ograniczenia przeprowadzonych badań oraz możliwe kierunki dalszego rozwoju i optymalizacji zaimplementowanych algorytmów.

Bibliografia

- [1] Cormen T.H., Leiserson C.E., Rivest R.L., Stein C., *Introduction to Algorithms*, 3rd Edition, The MIT Press, 2009.
- [2] Božidar D., Dobravec T., *Comparison of parallel sorting algorithms*, Technical report, Faculty of Computer and Information Science, University of Ljubljana, November 2015.
- [3] Maus A., *A full parallel Quicksort algorithm for multicore processors*, Department of Informatics, University of Oslo.
- [4] Goodrich M.T., Tamassia R., *Algorithm Design and Applications*, Wiley, 2015.
- [5] Cormen T.H., Leiserson C.E., Rivest R.L., Stein C., *Introduction to Algorithms*, 4th Edition, The MIT Press, 2022.
- [6] Hong H., *Parallel Bucket Sorting Algorithm*, Computer Science Department, San Jose State University.
- [7] Grama A., Gupta A., Karypis G., Kumar V., *Introduction to Parallel Computing*, 2nd Edition, Addison-Wesley, 2003.
- [8] JáJá J., *An Introduction to Parallel Algorithms*, Addison-Wesley, 1992.
- [9] Hennessy J.L., Patterson D.A., *Computer Architecture: A Quantitative Approach*, 4th Edition, Morgan Kaufmann.
- [10] Intel, *Hyper-Threading Technology*, <https://www.intel.com/content/www/us/en/gaming/resothreading.html>, stan na dzień: 08.08.2026.
- [11] Python Software Foundation, *The Python Tutorial*, <https://docs.python.org/3/tutorial/index.html>, stan na dzień: 11.08.2026.
- [12] Python Software Foundation, *multiprocessing — Process-based parallelism*, <https://docs.python.org/3/library/multiprocessing.html>, stan na dzień: 11.08.2026.
- [13]

- [14] NumPy Developers, *NumPy documentation*, <https://numpy.org/doc/stable/>, stan na dzień: 11.08.2026.
- [15] The pandas development team, *pandas documentation*, <https://pandas.pydata.org/docs/>, stan na dzień: 11.08.2026.
- [16] Matplotlib Development Team, *Matplotlib documentation*, <https://matplotlib.org/>, stan na dzień: 11.08.2026.
- [17] Rodola G., *psutil documentation*, <https://psutil.readthedocs.io/en/latest/>, stan na dzień: 11.08.2026.
- [18] Fredrikson B., *py-spy: Sampling profiler for Python programs*, <https://github.com/benfred/py-spy>, stan na dzień: 11.08.2026.
- [19] JetBrains, *PyCharm – The Python IDE*, <https://www.jetbrains.com/pycharm/>, stan na dzień: 11.08.2026.
- [20] Hipp D.R., *About SQLite*, <https://sqlite.org/about.html>, stan na dzień: 11.08.2026.

Spis rysunków

Spis tabel

Spis listingów

3.1	Funkcja partycjonowania w Quicksort	13
3.2	Tworzenie procesów w Parallel Quicksort	14
3.3	Funkcja scalania w Merge Sort	15
3.4	Równoległe sortowanie połówek i sekwencyjne scalanie w Parallel Merge Sort	16
3.5	Rozdzielanie elementów do kubków w Bucket Sort	16
3.6	Przydzielanie grup kubków procesom w Parallel Bucket Sort	17
3.7	Rozdzielanie elementów do kubków na podstawie wartości rozdzielających w Samplesort	18
3.8	Równoległe sortowanie grup kubków w Parallel Samplesort	19
3.9	Generowanie danych częściowo posortowanych	20

Streszczenie

W niniejszej pracy przeprowadzono analizę wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach. W ramach pracy zaimplementowano i porównano algorytmy Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort w wariantach sekwencyjnych i równoległych.

Badania obejmowały pomiary czasu wykonania, zużycia pamięci operacyjnej oraz obciążenia procesora dla różnych rozmiarów danych, liczby rdzeni oraz charakterystyk zbiorów wejściowych. Na podstawie uzyskanych wyników przeanalizowano skalowalność algorytmów oraz wyznaczono wartości przyspieszenia (*speedup*) i efektywności (*efficiency*).

Przeprowadzone eksperymenty pozwoliły na ocenę wpływu równoległości na wydajność poszczególnych algorytmów oraz identyfikację czynników ograniczających ich efektywność, takich jak narzut związany z tworzeniem procesów, synchronizacją, komunikacją i zarządzaniem pamięcią.

Summary

This thesis presents an analysis of the performance and efficiency of selected parallel sorting algorithms on multicore processors. The work implements and compares Parallel Quicksort, Parallel MergeSort, Parallel BucketSort, and Parallel Samplesort in both sequential and parallel variants.

The conducted experiments include measurements of execution time, memory usage, and processor utilization for different data sizes, numbers of processor cores, and characteristics of input datasets. Based on the obtained results, the scalability of the algorithms is analyzed and the speedup and efficiency metrics are determined.

The experiments make it possible to assess the impact of parallelization on the performance of the selected algorithms and to identify factors limiting their efficiency, including the overhead associated with process creation, synchronization, communication, and memory management.

Słowa kluczowe

Algorytmy sortowania, sortowanie równoległe, procesory wielordzeniowe, Quick-sort, Merge Sort, Bucket Sort, Samplesort, programowanie równoległe, wydajność, efektywność, skalowalność, Python, multiprocessing

Imię i nazwisko: Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Wydział: Informatyki i Sztucznej Inteligencji

Oświadczenie autora pracy dyplomowej*

Oświadczam pod rygorem odpowiedzialności karnej, że złożona przeze mnie praca dyplomowa pt. „*Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach*” jest moim samodzielnym opracowaniem i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Jednocześnie oświadczam, że moja praca (w całości ani we fragmentach) nie była wcześniej przedmiotem procedur związanych z uzyskaniem tytułu zawodowego w Politechnice Częstochowskiej.

Wyrażam/~~nie wyrażam~~** zgodę/zgody na nieodpłatne wykorzystanie przez Politechnikę Częstochowską całości lub fragmentów wyżej wymienionej pracy w publikacjach Politechniki Częstochowskiej.

.....
podpis studenta

*W przypadku zbiorowej pracy dyplomowej, dołącza się oświadczenia każdego ze współautorów pracy dyplomowej.

*Niepotrzebne skreślić.